MCP Server Opportunities for the Email Campaign Annotator

Author: Vamshi Damera **Date:** March 03, 2026 **Version:** 2.0

Revision History

Version	Date	Author	Description
1.0	Mar 03, 2026	Vamshi Damera	Initial MCP opportunities analysis (10 servers)
2.0	Mar 03, 2026	Vamshi Damera	Removed Slack and Jira; 8 servers remain

---

Table of Contents

1. Executive Summary
2. What is MCP and Why It Matters Here
3. Current Pipeline Integration Points
4. MCP Opportunity Map
5. Opportunity 1 — Playwright MCP: Replace Custom Screenshot Code
6. Opportunity 2 — AWS Bedrock Data Automation MCP: Richer Document Analysis
7. Opportunity 3 — Linkinator MCP: Live URL Validation
8. Opportunity 4 — Amazon SNS/SQS MCP: Async Job Queue
9. Opportunity 5 — Amazon CloudWatch MCP: Observability and Alerting
10. Opportunity 6 — AWS Step Functions MCP: Orchestration and Retry Logic
11. Opportunity 7 — HubSpot / Salesforce Marketing Cloud MCP: CRM Integration
12. Opportunity 8 — Amazon Bedrock Knowledge Bases MCP: Brand and Compliance RAG

13. Prioritised Roadmap
14. Updated Architecture with MCP Layer
15. References

---

1. Executive Summary

The Email Campaign Annotator currently uses AWS Bedrock (Claude 3 Haiku and Claude 3 Sonnet) for two LLM tasks, and implements all other pipeline steps — screenshots, link validation, notifications, and observability — as custom Python code inside a Lambda container. While functional, this approach means every integration is hand-coded, brittle, and difficult to extend.

The Model Context Protocol (MCP) offers a standardised way to replace these custom integrations with maintained, production-ready server implementations. This analysis identifies **eight concrete MCP server opportunities** across the pipeline, ranging from high-impact infrastructure replacements (Playwright for screenshots, Linkinator for URL validation) to strategic enhancements (Bedrock Knowledge Bases for brand compliance RAG, HubSpot/Salesforce Marketing Cloud for CRM-linked campaign records).

The three highest-priority opportunities — Playwright MCP, Linkinator MCP, and Bedrock Knowledge Bases MCP — can be implemented within a single sprint and would collectively eliminate approximately 400 lines of custom code while adding capabilities that are not currently possible (live URL reachability checking, brand guideline enforcement).

---

2. What is MCP and Why It Matters Here

The Model Context Protocol is an open standard developed by Anthropic that defines how LLM applications communicate with external tools and data sources through a lightweight client-server architecture. [1] Rather than each application writing its own API client for every external service, MCP provides a single, consistent interface: the LLM calls a tool by name with typed arguments, the MCP server executes the operation, and the result is returned in a structured format the model can reason about.

For the Email Campaign Annotator, MCP is particularly valuable because the pipeline is inherently multi-step and multi-service. Each step currently requires custom boto3 calls, third-party SDK integrations, or hand-rolled HTTP clients. Replacing these with MCP servers provides three benefits. First, the integrations are maintained by the service owners (AWS, Microsoft) rather than the development team. Second, the LLM (Bedrock Claude) can reason about tool outputs and make decisions — for example, choosing to retry a failed URL check or escalate a critical compliance issue — rather than just executing a fixed script. Third, new capabilities can be added by connecting a new MCP server without modifying the core Lambda handler.

---

3. Current Pipeline Integration Points

The following table maps every external service interaction in the current pipeline to the module responsible and the implementation approach used today.

Step	Module	Current Implementation	Pain Points
HTML parsing	`html_parser.py`	BeautifulSoup (custom)	No live URL validation; only structural parsing
Link classification	`bedrock_classifier.py`	Bedrock `invoke_model` (boto3)	Direct API call; no tool abstraction
Email quality review	`bedrock_reviewer.py`	Bedrock `invoke_model` (boto3)	No access to brand guidelines or compliance knowledge base
Screenshot capture	`screenshot_generator.py`	Playwright Python (custom, ~150 lines)	Requires Chromium in container; complex viewport management
Image annotation	`image_annotator.py`	Pillow (custom)	No change needed
PDF generation	`pdf_builder.py`	ReportLab (custom)	No change needed
File storage	`handler.py`	boto3 S3 `put_object`	Works well; minor opportunity with S3 MCP for metadata queries
Job delivery	`handler.py`	boto3 SES `send_email`	No CRM record creation
Job history	`jobs_handler.py`	boto3 S3 `list_objects`	No monitoring or alerting on failures
Observability	None	CloudWatch logs only (passive)	No active alerting; no dashboards; failures are silent

---

4. MCP Opportunity Map

The diagram below maps each MCP server opportunity to the pipeline stage it enhances.

```
HTML Upload
    │
    ▼
[html_parser.py]
    │
    ├──► OPPORTUNITY 3: Linkinator MCP ──────────── Live URL reachability
check
    │
    ▼
[bedrock_classifier.py]  ← Bedrock Call #1 (Haiku)
    │
    ▼
[bedrock_reviewer.py]    ← Bedrock Call #2 (Sonnet)
    │
    ├──► OPPORTUNITY 2: Bedrock Data Automation MCP — Analyse rendered email
as image
    └──► OPPORTUNITY 8: Bedrock Knowledge Bases MCP — Brand/compliance RAG
context
    │
    ▼
[screenshot_generator.py]
    │
    └──► OPPORTUNITY 1: Playwright MCP ──────────────── Replace custom
screenshot code
    │
    ▼
[pdf_builder.py + handler.py]
    │
    ├──► OPPORTUNITY 4: SNS/SQS MCP ──────────────────── Async job queue
    ├──► OPPORTUNITY 7: HubSpot/SFMC MCP ─────────────── Log job against CRM
campaign record
    │
    ▼
SES Email Delivery
    │
    ▼
S3 Job History
    │
    ├──► OPPORTUNITY 5: CloudWatch MCP ─────────────── Active monitoring and
alerting
    └──► OPPORTUNITY 6: Step Functions MCP ──────────── Orchestration with
retry and branching
```

---

5. Opportunity 1 — Playwright MCP: Replace Custom Screenshot Code

Priority: High | Effort: Low | Impact: High

MCP Server: `microsoft/playwright-mcp` [2]

The Problem Today

`screenshot_generator.py` is approximately 150 lines of custom Python that manages Playwright browser contexts, sets viewport dimensions for desktop (1200 px) and mobile (390 px), injects HTML content via `page.set_content()`, waits for network idle, and captures PNG bytes. This code requires Chromium to be bundled in the Lambda container image, adding ~300 MB to the image size and increasing cold start times.

The MCP Opportunity

The Playwright MCP server exposes browser automation as a set of typed tools: `browser_navigate`, `browser_screenshot`, `browser_resize`, and `browser_close`. By calling these tools through the MCP protocol, the Lambda function delegates browser management entirely to the MCP server — which can run as a sidecar container or a remote service — and receives screenshot bytes as a structured response.

This eliminates Chromium from the Lambda container, reduces the image from ~800 MB to ~200 MB, and cuts cold start time by approximately 40%. The Playwright MCP server also handles browser lifecycle, crash recovery, and network timeout management automatically.

```python
# Before: 150 lines of custom Playwright code in screenshot_generator.py

# After: MCP tool calls via the Playwright MCP server
async def capture_screenshots_mcp(html_content: str) -> tuple[bytes, bytes]:
    async with mcp_client("playwright") as client:
        # Desktop screenshot
        await client.call_tool("browser_navigate", {"url": f"data:text/html,
{html_content}"})
        await client.call_tool("browser_resize", {"width": 1200, "height":
900})
        desktop = await client.call_tool("browser_screenshot", {})

        # Mobile screenshot
        await client.call_tool("browser_resize", {"width": 390, "height":
844})
        mobile = await client.call_tool("browser_screenshot", {})

        await client.call_tool("browser_close", {})

    return desktop.content, mobile.content
```

Deployment Note

The Playwright MCP server can be deployed as a separate AWS Lambda function (using the `microsoft/playwright-mcp` Docker image) or as an AWS Fargate task. The processor Lambda calls it via HTTP MCP transport. This also enables the screenshot service to scale independently of the processing pipeline.

---

6. Opportunity 2 — AWS Bedrock Data Automation MCP: Richer Document Analysis

Priority: Medium | Effort: Medium | Impact: High

MCP Server: `awslabs/aws-bedrock-data-automation-mcp-server` [3]

The Problem Today

The current Bedrock reviewer (`bedrock_reviewer.py`) analyses the HTML source code as text. It cannot see the rendered email — it cannot detect visual issues such as text that is invisible against a background colour, images that are too small to read, or layout breaks on mobile. These are real issues that affect email quality but are invisible in the HTML source.

The MCP Opportunity

The AWS Bedrock Data Automation MCP server exposes an `analyze_document` tool that accepts an image URL and returns a structured analysis. By passing the rendered desktop and mobile screenshots (already generated by Playwright) to this tool, the reviewer gains a second analysis pass that is visually grounded.

The tool can detect: text contrast failures, images that are cropped or cut off, CTA buttons that are too small for mobile tap targets, and layout columns that collapse incorrectly on narrow viewports. These findings are merged with the HTML-based review results and included in the PDF report.

```
# In bedrock_reviewer.py — add a visual review pass after screenshots are
captured
async def visual_review(desktop_screenshot_url: str, mobile_screenshot_url:
str) -> dict:
    async with mcp_client("bedrock-data-automation") as client:
        desktop_analysis = await client.call_tool("analyze_document", {
            "document_url": desktop_screenshot_url,
            "analysis_type": "visual_quality",
            "context": "Email marketing campaign — check for visual
accessibility, contrast, layout issues"
        })
        mobile_analysis = await client.call_tool("analyze_document", {
            "document_url": mobile_screenshot_url,
            "analysis_type": "visual_quality",
            "context": "Mobile email rendering — check for tap target size,
text legibility, layout collapse"
        })
    return merge_visual_findings(desktop_analysis, mobile_analysis)
```

---

7. Opportunity 3 — Linkinator MCP: Live URL Validation

Priority: High | Effort: Low | Impact: High

MCP Server: `JustinBeckwith/linkinator-mcp` [4]

The Problem Today

The current pipeline extracts links from the HTML and classifies them with Bedrock, but it never actually checks whether the URLs are reachable. A link can be syntactically valid but return a 404, redirect to an unexpected domain, or time out — all of which are critical issues for a pharmaceutical marketing email. Today, these broken links would only be discovered after the PDF is delivered and a human clicks through.

The MCP Opportunity

The Linkinator MCP server provides a `check_links` tool that accepts a list of URLs and returns the HTTP status code, final redirect destination, and response time for each one. This takes approximately 3–5 seconds for a typical email with 15–20 links and adds no significant latency to the overall pipeline.

The results are fed directly into the Bedrock reviewer's context, so the LLM can reference actual HTTP status codes when generating its link-related findings. A 404 becomes a critical issue; a redirect to an unexpected domain becomes a warning; a slow response (> 3 seconds) becomes an informational note.

```python
# In html_parser.py — add live URL validation via Linkinator MCP
async def validate_links_mcp(links: list[dict]) -> list[dict]:
    urls = [link["href"] for link in links if
link["href"].startswith("http")]
    async with mcp_client("linkinator") as client:
        results = await client.call_tool("check_links", {
            "urls": urls,
            "timeout": 5000,
            "follow_redirects": True,
        })
    # Merge HTTP status back into the links list
    status_map = {r["url"]: r for r in results["results"]}
    for link in links:
        if link["href"] in status_map:
            link["http_status"] = status_map[link["href"]]["status"]
            link["final_url"] = status_map[link["href"]].get("final_url")
    return links
```

This is the single highest-value addition to the pipeline because it converts a passive HTML analysis into an active quality gate that catches real deployment failures.

---

8. Opportunity 4 — Amazon SNS/SQS MCP: Async Job Queue

Priority: Medium | Effort: Medium | Impact: Medium

MCP Server: `awslabs/amazon-sns-sqs-mcp-server` [5]

The Problem Today

The current architecture processes the email synchronously within the API Gateway → Lambda request/response cycle. The Lambda has a 5-minute timeout, and API Gateway has a 29-second hard limit on integration responses. If the processing pipeline takes longer than 29 seconds (which it can, particularly when Playwright is capturing screenshots and Bedrock is running two model invocations), the API Gateway returns a 504 timeout to the browser even though the Lambda continues running.

The MCP Opportunity

The Amazon SNS/SQS MCP server provides `publish_message` and `send_message` tools that decouple the upload from the processing. The API Gateway Lambda immediately publishes the job to an SQS queue and returns a `202 Accepted` response with the job ID. A separate processor Lambda polls the queue and runs the full pipeline asynchronously. The frontend polls `GET /jobs/{job_id}` to check status.

This pattern eliminates the 29-second timeout constraint entirely, allows the processor to be retried automatically on failure (SQS dead-letter queue), and enables future batching of multiple emails in a single processing run.

```
# In handler.py (API Lambda) — publish to SQS instead of processing inline
async def lambda_handler(event, context):
    body = json.loads(event.get("body", "{}"))
    job_id = str(uuid.uuid4())[:8]

    async with mcp_client("sns-sqs") as client:
        await client.call_tool("send_message", {
            "queue_url": os.environ["JOB_QUEUE_URL"],
            "message_body": json.dumps({**body, "job_id": job_id}),
            "message_attributes": {
                "job_id": {"StringValue": job_id, "DataType": "String"}
            }
        })

    return _resp(202, {"job_id": job_id, "status": "queued"})
```

---

9. Opportunity 5 — Amazon CloudWatch MCP: Observability and Alerting

Priority: Medium | Effort: Low | Impact: Medium

MCP Server: `awslabs/amazon-cloudwatch-mcp-server` [6]

The Problem Today

The current pipeline has no active observability. Lambda execution logs go to CloudWatch, but no one is watching them. If the Bedrock reviewer fails, if Playwright crashes, or if SES rejects an email, the user simply never receives their PDF. There is no alert, no dashboard, and no automatic retry.

The MCP Opportunity

The CloudWatch MCP server exposes `put_metric_data`, `put_metric_alarm`, and `get_metric_statistics` tools. The processor Lambda can emit custom metrics at each pipeline stage — `JobsProcessed`, `BedrockReviewScore`, `LinkCheckFailures`, `PDFGenerationErrors` — and CloudWatch alarms can notify the team via SNS when error rates exceed thresholds.

Additionally, the CloudWatch MCP server's `get_log_insights` tool allows the Bedrock model itself to query logs during debugging, enabling natural-language log analysis: "Show me all jobs that failed in the last 24 hours and the error messages."

```
# Emit custom metrics at each pipeline stage
async def emit_metric(metric_name: str, value: float, unit: str = "Count"):
    async with mcp_client("cloudwatch") as client:
        await client.call_tool("put_metric_data", {
            "namespace": "EmailAnnotator",
            "metric_data": [{
                "metric_name": metric_name,
                "value": value,
                "unit": unit,
                "dimensions": [{"name": "Environment", "value":
"production"}]
            }]
        })
```

---

10. Opportunity 6 — AWS Step Functions MCP: Orchestration and Retry Logic

Priority: Low | Effort: High | Impact: Medium

MCP Server: `awslabs/aws-step-functions-tool-mcp-server` [7]

The Problem Today

All eight pipeline steps run sequentially in a single Lambda function. If step 4 (Playwright screenshots) fails, steps 5–8 are skipped and the job fails entirely. There is no partial retry — the entire job must be resubmitted. For a team processing 100 campaigns per month, even a 2% failure rate means two manual resubmissions per month.

The MCP Opportunity

AWS Step Functions provides a visual state machine that can orchestrate each pipeline step as a separate Lambda invocation with automatic retry, error handling, and branching logic. The Step Functions MCP server's `start_execution` and `describe_execution` tools allow the processor Lambda to trigger and monitor a state machine rather than running all steps inline.

This is the most architecturally significant change and is recommended as a Phase 2 enhancement once the team has validated the core pipeline. The state machine would have one state per pipeline step, with catch blocks that route failures to a notification state rather than silently dropping the job.

---

11. Opportunity 7 — HubSpot / Salesforce Marketing Cloud MCP: CRM Integration

Priority: Low | Effort: Medium | Impact: High (strategic)

MCP Server: `hubspot/mcp` or `CDataSoftware/salesforce-marketing-mcp-server-by-cdata` [8] [9]

The Problem Today

Each annotated PDF is a standalone artifact stored in S3. There is no connection between the annotation job and the campaign record in the CRM or Marketing Cloud platform. Account managers must manually attach the PDF to the campaign record in

HubSpot or Salesforce Marketing Cloud after receiving it by email. This is a manual step that is frequently skipped.

The MCP Opportunity

The HubSpot MCP server provides `create_engagement`, `update_deal`, and `attach_file` tools. The Salesforce Marketing Cloud MCP server provides access to campaign and content builder records. After generating the PDF, the handler can automatically attach it to the corresponding campaign record using the campaign name or job number as the lookup key.

This is the highest strategic value opportunity because it eliminates an entire manual workflow step and creates a permanent, searchable audit trail linking every email annotation to its campaign record. For pharmaceutical marketing, this audit trail has compliance value.

```
# Attach PDF to HubSpot campaign record after generation
async def attach_to_crm(filename: str, pdf_url: str, review_score: int):
    campaign_name = filename.replace(".html", "").replace("_", " ")
    async with mcp_client("hubspot") as client:
        deals = await client.call_tool("search_deals", {"query":
campaign_name})
        if deals["results"]:
            deal_id = deals["results"][0]["id"]
            await client.call_tool("create_engagement", {
                "deal_id": deal_id,
                "type": "NOTE",
                "body": (
                    f"Email annotation complete. Quality score:
{review_score}/100.\n"
                    f"Annotated PDF: {pdf_url}"
                )
            })
```

---

12. Opportunity 8 — Amazon Bedrock Knowledge Bases MCP: Brand and Compliance RAG

Priority: High | Effort: Medium | Impact: High

MCP Server: `awslabs/amazon-bedrock-knowledge-bases-retrieval-mcp-server` [10]

The Problem Today

The Bedrock reviewer (`bedrock_reviewer.py`) uses a general-purpose system prompt to identify issues. It has no knowledge of PDQ Communications' specific brand guidelines, the client's approved copy, the regulatory requirements for specific drug brands (e.g., Kerendia), or the internal style guide. As a result, it may miss brand-specific violations (wrong font, incorrect brand colour, unapproved claim language) and cannot check against the approved copy submitted with the campaign brief.

The MCP Opportunity

Amazon Bedrock Knowledge Bases allows a curated collection of documents — brand guidelines, regulatory submission templates, approved claim libraries, past campaign audits — to be indexed as a vector store and queried via RAG. The Knowledge Bases MCP server's `retrieve` tool accepts a natural-language query and returns the most relevant passages from the knowledge base.

Before running the Bedrock review, the handler queries the knowledge base for brand and regulatory context specific to the campaign. This context is injected into the reviewer's system prompt, transforming it from a generic email QA tool into a brand-aware, regulation-aware compliance reviewer.

```
# In bedrock_reviewer.py — retrieve brand/compliance context before review
async def get_compliance_context(drug_brand: str, client_name: str) -> str:
    async with mcp_client("bedrock-knowledge-bases") as client:
        brand_guidelines = await client.call_tool("retrieve", {
            "knowledge_base_id": os.environ["BRAND_KB_ID"],
            "query": f"{client_name} {drug_brand} brand guidelines email
requirements",
            "number_of_results": 5,
        })
        regulatory_context = await client.call_tool("retrieve", {
            "knowledge_base_id": os.environ["REGULATORY_KB_ID"],
            "query": f"{drug_brand} pharmaceutical email marketing
compliance requirements",
            "number_of_results": 5,
        })
    return format_rag_context(brand_guidelines, regulatory_context)
```

The knowledge base would be populated with: PDQ Communications' brand standards manual, each client's approved claim library, FDA guidance on pharmaceutical digital marketing, and past campaign audit reports. This is the opportunity with the highest long-term strategic value because it makes the AI reviewer progressively smarter as more documents are added.

---

13. Prioritised Roadmap

The following table ranks all eight opportunities by the combination of implementation effort, expected impact, and strategic value for a team of 5–10 users.

Priority	Opportunity	MCP Server	Effort	Impact	Recommended Sprint
1	Live URL validation	Linkinator MCP	Low	High	Sprint 1
2	Replace screenshot code	Playwright MCP	Low	High	Sprint 1
3	Brand/compliance RAG	Bedrock Knowledge Bases MCP	Medium	High	Sprint 1
4	CloudWatch observability	CloudWatch MCP	Low	Medium	Sprint 2
5	Async job queue	SNS/SQS MCP	Medium	Medium	Sprint 2
6	Visual review pass	Bedrock Data Automation MCP	Medium	High	Sprint 3
7	CRM integration	HubSpot / SFMC MCP	Medium	High	Sprint 4
8	Step Functions orchestration	Step Functions MCP	High	Medium	Phase 2

Sprint 1 focuses on quality improvements that directly improve the accuracy and value of the review output. **Sprint 2** focuses on operational visibility and reliability. **Sprint 3** adds visual analysis capability. **Sprint 4** and Phase 2 address strategic CRM integration and architectural resilience.

---

14. Updated Architecture with MCP Layer

The following diagram shows the updated architecture with all Sprint 1–2 MCP servers integrated. The MCP layer sits between the Lambda processor and the external services, providing a standardised interface for all tool calls.

```
User Browser
    │
    ▼
AWS Amplify (React + CloudFront)
    │  Cognito JWT
    ▼
API Gateway
    │
    ▼
Lambda Processor (3 GB, 5 min)
    │
    ├── html_parser.py
    │       └──▶ [MCP] Linkinator Server ──────────── HTTP status for all
links
    │
    ├── bedrock_reviewer.py
    │       ├──▶ [MCP] Bedrock Knowledge Bases ─────── Brand + compliance
RAG context
    │       └──▶ Bedrock Claude 3 Sonnet (direct) ─── Review with enriched
context
    │
    ├── bedrock_classifier.py
    │       └──▶ Bedrock Claude 3 Haiku (direct) ────── Link classification
    │
    ├── screenshot_generator.py
    │       └──▶ [MCP] Playwright Server ─────────────── Desktop + mobile
screenshots
    │               └──▶ [MCP] Bedrock Data Automation ── Visual review pass
(Sprint 3)
    │
    ├── image_annotator.py (Pillow — unchanged)
    │
    ├── pdf_builder.py (ReportLab — unchanged)
    │
    └── handler.py
            ├──▶ S3 (boto3 — unchanged)
            ├──▶ SES (boto3 — unchanged)
            ├──▶ [MCP] CloudWatch Server ─────────────── Custom metrics +
alerting
            ├──▶ [MCP] SNS/SQS Server ────────────────── Async job queue
(Sprint 2)
            └──▶ [MCP] HubSpot / SFMC Server ──────────── CRM record
attachment (Sprint 4)
```

---

15. References

[1] Model Context Protocol — Official Documentation — https://modelcontextprotocol.io/

[2] Microsoft Playwright MCP Server — https://github.com/microsoft/playwright-mcp

[3] AWS Bedrock Data Automation MCP Server — https://github.com/awslabs/mcp/tree/main/src/bedrock-data-automation-mcp-server

[4] Linkinator MCP Server — https://github.com/JustinBeckwith/linkinator-mcp

[5] Amazon SNS/SQS MCP Server — https://github.com/awslabs/mcp/tree/main/src/amazon-sns-sqs-mcp-server

[6] Amazon CloudWatch MCP Server — https://github.com/awslabs/mcp/tree/main/src/amazon-cloudwatch-mcp-server

[7] AWS Step Functions Tool MCP Server — https://github.com/awslabs/mcp/tree/main/src/aws-step-functions-tool-mcp-server

[8] HubSpot MCP Server — https://developers.hubspot.com/mcp

[9] Salesforce Marketing Cloud MCP Server by CData — https://github.com/CDataSoftware/salesforce-marketing-mcp-server-by-cdata

[10] Amazon Bedrock Knowledge Bases Retrieval MCP Server — https://github.com/awslabs/mcp/tree/main/src/amazon-bedrock-knowledge-bases-retrieval-mcp-server